

Note technique - Gestion fermée des utilisateurs dans G2M

1. Objet

Cette note technique décrit une proposition d'implémentation progressive d'un système de gestion fermée des utilisateurs pour G2M. Le principe retenu est le suivant : aucun utilisateur ne peut s'inscrire librement. Un compte ne peut être activé que si un administrateur a préalablement créé une invitation valide pour l'adresse email concernée.

Le dispositif couvre :

- la création d'invitations par l'administrateur ;
- l'envoi ou le renvoi du lien d'activation lorsqu'un utilisateur invité tente de se connecter avec son email ;
- l'activation du compte par l'utilisateur invité ;
- le hachage sécurisé du mot de passe ;
- la connexion ;
- la protection des routes ;
- la gestion des rôles ;
- une base d'habilitations par rôle et par utilisateur ;
- la journalisation minimale des actions sensibles.

2. Étape 1 - Analyse de l'existant

2.1. Structure actuelle observée

Le projet G2M est organisé autour d'une structure Express classique :

Zone	Fichiers ou dossiers identifiés	Rôle actuel
Point d'entrée Express	<code>app.js</code>	Configuration Express, EJS, fichiers statiques, routes principales, i18n, erreurs
Base de données	<code>config/database.js</code>	Initialisation SQLite avec <code>better-sqlite3</code> , création des tables principales
Routes utilisateurs	<code>routes/userRoutes.js</code>	Liste, création, détail, édition des utilisateurs
Contrôleur utilisateurs	<code>controllers/userController.js</code>	Logique de gestion administrative des utilisateurs
Modèle utilisateur	<code>models/User.js</code>	Accès SQLite à la table <code>users</code> et aux régions affectées
Modèle rôles	<code>models/Role.js</code>	Lecture des rôles depuis la table <code>roles</code>

Vues utilisateurs	<code>views/users/index.ejs</code> , <code>views/users/form.ejs</code> , <code>views/users/show.ejs</code>	UI actuelle de gestion des utilisateurs
Locales	<code>locales/fr.json</code> , <code>locales/en.json</code> , <code>locales/es.json</code>	Libellés i18n déjà utilisés dans les vues
Tests	<code>tests/app.test.js</code>	Tests fonctionnels Node.js avec <code>node:test</code> et <code>supertest</code>

2.2. État actuel du module utilisateurs

Le module `users` existe déjà. Il permet :

- de lister les utilisateurs ;
- de créer un utilisateur ;
- de modifier un utilisateur ;
- d'affecter des régions ;
- d'associer un rôle ;
- de gérer un statut simple.

La table `users` contient actuellement :

- `id` ;
- `nom` ;
- `prenoms` ;
- `email` ;
- `telephone` ;
- `role` ;
- `statut` ;
- `password_hash` ;
- `last_login` ;
- `created_at`.

Les statuts existants sont actuellement `actif`, `inactif`, `suspendu`. Le besoin cible demande `invité`, `actif`, `suspendu`, `désactivé`. Il faudra donc migrer prudemment la colonne `statut`, en remplaçant progressivement `inactif` par `desactive`.

2.3. Écarts par rapport au besoin cible

Les éléments suivants ne sont pas encore présents :

- table `user_invitations` ;
- table `activation_tokens` ;
- table `audit_logs` ou `login_logs` ;
- génération de token d'activation ;

- activation par lien ;
- définition initiale du mot de passe ;
- hachage bcrypt réellement utilisé ;
- page de connexion ;
- middleware `requireAuth` ;
- middleware `requireRole` ;
- gestion JWT ou session ;
- habilitations par rôle ou par utilisateur ;
- protection des routes existantes.

2.4. Hypothèses techniques raisonnables

Pour respecter l'architecture G2M actuelle, les hypothèses suivantes sont retenues :

- conserver Express, EJS et SQLite ;
- ne pas remplacer le module `users`, mais l'étendre ;
- utiliser `bcrypt` pour le hachage ;
- utiliser `jsonwebtoken` avec un cookie `HttpOnly` pour l'authentification, car le contexte projet mentionne JWT ;
- prévoir une compatibilité future PostgreSQL en évitant les particularités SQLite non portables ;
- stocker les dates au format ISO texte, comme le projet le fait déjà ;
- garder les libellés UI dans les fichiers `locales/*`.

2.5. Plan d'intégration proposé

L'intégration doit se faire en étapes successives :

1. Étendre le schéma SQLite et les rôles.
2. Ajouter les modèles `UserInvitation`, `ActivationToken`, `AuditLog`, `Permission`.
3. Adapter `User` pour gérer statut, mot de passe, email vérifié et dernière connexion.
4. Ajouter les routes administrateur d'invitation.
5. Ajouter le parcours public d'activation.
6. Ajouter la connexion JWT en cookie sécurisé.
7. Protéger progressivement les routes.
8. Ajouter l'écran de gestion des habilitations.
9. Ajouter les tests automatiques et préparer les tests manuels.

3. Étape 2 - Modèle de données

3.1. Table `users`

La table existante doit être étendue plutôt que recrée brutalement.

Structure cible :

```
CREATE TABLE users ( id INTEGER PRIMARY KEY AUTOINCREMENT, nom TEXT NOT NULL,
prenoms TEXT NOT NULL, email TEXT NOT NULL COLLATE NOCASE UNIQUE, telephone TEXT,
password_hash TEXT, role TEXT NOT NULL DEFAULT 'partenaire', zone_affectation
TEXT, mission_id INTEGER, statut TEXT NOT NULL DEFAULT 'invite' CHECK (statut
IN ('invite', 'actif', 'suspendu', 'desactive')), email_verified INTEGER NOT NULL
DEFAULT 0, last_login TEXT, created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (mission_id)
REFERENCES missions(id) ON UPDATE CASCADE ON DELETE SET NULL );
```

Remarque : le libellé affiché peut être `invité` et `désactivé`, mais les codes techniques doivent rester ASCII : `invite`, `desactive`. Cela évite les problèmes de comparaison, d'URL, de migration et de compatibilité PostgreSQL.

3.2. Table `user_invitations`

Cette table porte l'invitation administrative.

```
CREATE TABLE user_invitations ( id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT
NOT NULL COLLATE NOCASE UNIQUE, nom TEXT NOT NULL, prenoms TEXT NOT NULL, role
TEXT NOT NULL, zone_affectation TEXT, mission_id INTEGER, statut TEXT NOT NULL
DEFAULT 'invite' CHECK (statut IN ('invite', 'activee', 'expiree', 'annulee')),
invited_by INTEGER, invitation_token_hash TEXT NOT NULL, expires_at TEXT NOT NULL,
activated_at TEXT, created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at
TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (mission_id) REFERENCES
missions(id) ON UPDATE CASCADE ON DELETE SET NULL, FOREIGN KEY (invited_by)
REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL );
```

3.3. Table `activation_tokens`

Cette table permet de gérer l'usage unique des tokens. Elle peut sembler redondante avec `user_invitations.invitation_token_hash`, mais elle prépare mieux les usages futurs : renouvellement, expiration, révocation, reset password.

```
CREATE TABLE activation_tokens ( id INTEGER PRIMARY KEY AUTOINCREMENT,
invitation_id INTEGER NOT NULL, user_id INTEGER, token_hash TEXT NOT NULL UNIQUE,
purpose TEXT NOT NULL DEFAULT 'activation' CHECK (purpose IN ('activation',
'password_reset')), expires_at TEXT NOT NULL, used_at TEXT, created_at TEXT NOT
NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (invitation_id) REFERENCES
user_invitations(id) ON UPDATE CASCADE ON DELETE CASCADE, FOREIGN KEY
(user_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE );
```

3.4. Table `audit_logs`

Une table unique `audit_logs` est préférable à `login_logs` seule, car elle pourra enregistrer les connexions, invitations, suspensions et modifications d'habilitations.

```
CREATE TABLE audit_logs ( id INTEGER PRIMARY KEY AUTOINCREMENT, actor_user_id
INTEGER, target_user_id INTEGER, action TEXT NOT NULL, entity_type TEXT,
entity_id TEXT, ip_address TEXT, user_agent TEXT, details_json TEXT,
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (actor_user_id)
REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL, FOREIGN KEY
(target_user_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET NULL );
```

3.5. Tables d'habilitations

Pour l'étape 7, un modèle simple et évolutif est recommandé.

```
CREATE TABLE permissions ( id INTEGER PRIMARY KEY AUTOINCREMENT, code_permission
TEXT NOT NULL UNIQUE, label TEXT NOT NULL, description TEXT, created_at TEXT NOT
```

```

NULL DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE role_permissions ( role TEXT NOT
NULL, permission_id INTEGER NOT NULL, allowed INTEGER NOT NULL DEFAULT 1,
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, PRIMARY KEY (role,
permission_id), FOREIGN KEY (permission_id) REFERENCES permissions(id) ON UPDATE
CASCADE ON DELETE CASCADE ); CREATE TABLE user_permission_overrides ( user_id
INTEGER NOT NULL, permission_id INTEGER NOT NULL, allowed INTEGER NOT NULL,
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, PRIMARY KEY (user_id,
permission_id), FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASCADE
ON DELETE CASCADE, FOREIGN KEY (permission_id) REFERENCES permissions(id) ON
UPDATE CASCADE ON DELETE CASCADE );

```

3.6. Rôles cibles

Les rôles techniques recommandés sont :

Code technique	Libellé
admin	Administrateur
coordonateur	Coordonateur national
superviseur	Superviseur régional
controleur	Contrôleur qualité
specialiste_gis	Responsable SIG
partenaire	Partenaire / bailleur en lecture seule

Le code `agent` existe déjà dans G2M pour les enquêteurs. Il peut être conservé pour les agents de collecte si nécessaire, mais il ne fait pas partie de la liste cible des profils de pilotage.

4. Étape 3 - Routes administrateur

4.1. Routes à créer ou adapter

Créer un module dédié est préférable pour ne pas mélanger la gestion historique des utilisateurs et le nouveau système d'invitation.

Fichiers proposés :

- `routes/adminUserRoutes.js`
- `controllers/adminUserController.js`
- `models/UserInvitation.js`
- `models/ActivationToken.js`
- `models/AuditLog.js`

Routes :

```

router.get("/users", requirePermission("users.read"), adminUserController.users);
router.get("/users/invitations", requirePermission("users.invite.read"),
adminUserController.invitations); router.get("/users/invitations/new",
requirePermission("users.invite.create"), adminUserController.newInvitation);
router.post("/users/invitations", requirePermission("users.invite.create"),
adminUserController.createInvitation); router.get("/users/invitations/:id/edit",
requirePermission("users.invite.update"), adminUserController.editInvitation);
router.post("/users/invitations/:id", requirePermission("users.invite.update"),
adminUserController.updateInvitation); router.post("/users/:id/suspend",
requirePermission("users.suspend"), adminUserController.suspendUser);
router.post("/users/:id/disable", requirePermission("users.disable"),
adminUserController.disableUser);

```

4.2. Choix d'intégration

Deux options existent :

- conserver `/users` comme page principale et y ajouter des onglets `Utilisateurs` et `Invitations` ;
- créer un espace `/admin/users`.

Recommandation : commencer avec `/users` pour préserver les conventions actuelles, puis introduire les onglets.

5. Étape 4 - Activation utilisateur

5.1. Parcours fonctionnel

10. L'administrateur crée une invitation.
11. G2M génère un token aléatoire en clair.
12. G2M ne stocke que le hash du token.
13. G2M peut envoyer immédiatement un email d'invitation, ou attendre que l'utilisateur tente une première connexion.
14. Si l'utilisateur saisit son email sur la page de connexion et que cet email correspond à une invitation valide non activée, G2M envoie ou renvoie un email contenant le lien d'activation.
15. Le lien d'activation est construit :

```
https://g2m.example.org/activation/<token>
```
16. L'utilisateur ouvre le lien.
17. G2M hash le token reçu et cherche une invitation valide.
18. L'utilisateur définit son mot de passe.
19. G2M crée ou active le compte.
20. Le token est marqué comme utilisé.
21. L'utilisateur est redirigé vers `/login`.

5.2. Déclenchement du lien depuis la page de connexion

Le comportement attendu lors d'une tentative de connexion est le suivant :

Situation	Comportement
Email inconnu	Afficher un message sobre : "Si votre adresse est autorisée, un lien d'activation ou de connexion vous sera envoyé." Aucun détail ne doit confirmer que l'email n'existe pas.
Email présent dans <code>user_invitations</code> , invitation valide, compte non activé	Générer ou réutiliser un token valide, envoyer l'email d'activation, afficher un message sobre.
Email invité mais invitation expirée	Afficher le même message sobre, journaliser l'échec et demander une nouvelle invitation côté administrateur.
Email déjà activé	Procéder à la vérification classique du mot de passe.
Compte suspendu ou désactivé	Refuser la connexion avec un message sobre, sans envoyer de nouveau lien d'activation.

Ce mécanisme ne constitue pas une inscription ouverte. L'utilisateur ne peut recevoir un lien que si son email figure déjà dans une invitation valide créée par l'administrateur.

5.3. Routes publiques

```
router.get("/activation/:token", activationController.show);
router.post("/activation/:token", activationController.activate);
```

5.4. Service d'envoi du lien d'activation

Un service dédié doit isoler l'envoi d'email :

```
services/
  mailService.js
  activationMailService.js
```

Pour le POC, deux modes peuvent coexister :

- mode développement : écrire le lien d'activation dans les logs serveur ;
- mode production : envoyer le mail via un fournisseur SMTP ou une API email.

Variables d'environnement à prévoir :

```
APP_BASE_URL=https://g2m.example.org
MAIL_FROM=no-reply@g2m.example.org
SMTP_HOST=
SMTP_PORT=
SMTP_USER=
SMTP_PASSWORD=
```

L'email envoyé doit rester simple :

- objet : Activation de votre compte G2M ;
- nom et prénoms de l'utilisateur invité ;
- lien d'activation ;
- date d'expiration ;
- mention indiquant d'ignorer le message si l'utilisateur n'est pas concerné.

5.5. Contrôles obligatoires

Le token doit être refusé si :

- il n'existe pas ;
- il est expiré ;
- il a déjà été utilisé ;
- l'invitation est annulée ou déjà activée ;
- un compte actif existe déjà pour cet email ;
- le mot de passe ne respecte pas la politique minimale.

6. Étape 5 - Authentification

6.1. Choix recommandé

Le contexte mentionne JWT. La solution recommandée est donc :

- JWT signé côté serveur ;
- stockage dans un cookie `HttpOnly`, `SameSite=Lax`, `Secure` en production ;
- durée de vie courte à moyenne, par exemple 8 heures ;
- middleware `currentUser` pour charger l'utilisateur depuis le token.

Cette solution reste simple pour Express/EJS et évite de manipuler le token dans le JavaScript frontend.

6.2. Routes d'authentification

```
router.get("/login", authController.loginForm); router.post("/login",  
authController.login); router.post("/logout", authController.logout);
```

6.3. Contrôle de connexion

La connexion doit :

- normaliser l'email en minuscules ;
- chercher l'utilisateur par email ;
- si aucun compte actif n'existe, chercher une invitation valide pour cet email ;
- si une invitation valide existe, envoyer ou renvoyer un lien d'activation ;
- vérifier que `statut = 'actif'` ;
- vérifier que `email_verified = 1` ;
- comparer le mot de passe via `bcrypt.compare` ;
- mettre à jour `last_login` ;
- écrire un log `auth.login_success` ou `auth.login_failed`.

6.4. Middlewares

Fichier proposé : `middlewares/authMiddleware.js`.

```
function requireAuth(req, res, next) { if (!req.currentUser) { return  
res.redirect(`/login?next=${encodeURIComponent(req.originalUrl)}`); } next(); }  
function requireRole(...roles) { return (req, res, next) => { if  
(!req.currentUser || !roles.includes(req.currentUser.role)) { return  
res.status(403).render("errors/403", { title: req.t("errors.403.title")  
}); } next(); }; }
```

7. Étape 6 - Sécurité minimale

7.1. Hachage des mots de passe

Installer `bcrypt` :

```
npm install bcrypt jsonwebtoken cookie-parser
```


Hachage :

```
const passwordHash = await bcrypt.hash(password, 12);
```

Vérification :

```
const ok = await bcrypt.compare(password, user.password_hash);
```

7.2. Token d'activation à usage unique

Le token en clair doit être généré avec `crypto.randomBytes`, envoyé à l'utilisateur, puis oublié.

```
const crypto = require("node:crypto"); function generateActivationToken() { return  
crypto.randomBytes(32).toString("hex"); } function hashToken(token) { return  
crypto.createHash("sha256").update(token).digest("hex"); }
```

7.3. Validation d'entrées

Les validations minimales :

- email conforme ;
- nom et prénoms obligatoires ;
- rôle existant ;
- statut autorisé ;
- mission existante si `mission_id` est renseigné ;
- mot de passe de longueur minimale, par exemple 10 caractères ;
- expiration cohérente.

7.4. Messages sobres

Pour éviter les fuites d'information :

- ne pas préciser si un email existe lors d'un échec de connexion ;
- afficher "Identifiants invalides ou compte non actif" ;
- afficher "Si votre adresse est autorisée, vous recevrez les instructions nécessaires" lorsqu'un email non activé est soumis ;
- afficher "Lien invalide ou expiré" pour l'activation ;
- journaliser le détail côté serveur ou dans `audit_logs`.

7.5. Journalisation

Actions à journaliser :

- invitation créée ;
- invitation modifiée ;
- email d'activation envoyé ;
- demande de renvoi de lien d'activation ;
- token utilisé ;
- activation réussie ;
- connexion réussie ;

- connexion échouée ;
- utilisateur suspendu ;
- utilisateur désactivé ;
- habilitation modifiée.

8. Étape 7 - Système d'habilitations par rôles et utilisateurs

8.1. Principe

Les rôles donnent des droits collectifs. Les overrides utilisateur permettent d'accorder ou retirer un droit à une personne précise.

Ordre de résolution recommandé :

22. si une dérogation utilisateur existe, elle prime ;
23. sinon, appliquer la permission du rôle ;
24. sinon, refuser.

8.2. Permissions de départ

Code permission	Description
dashboard.read	Accès au dashboard
missions.read	Consultation des missions
missions.manage	Création et modification des missions
teams.manage	Gestion des équipes
agents.manage	Gestion des agents
users.read	Consultation des utilisateurs
users.invite.create	Création d'invitations
users.invite.update	Modification d'invitations
users.suspend	Suspension de comptes
users.disable	Désactivation de comptes
kobo.manage	Administration KoboToolbox
sig.read	Accès à la cartographie
sig.manage	Paramétrage SIG
infographics.read	Consultation des infographies

8.3. Middleware `requirePermission`

```
function requirePermission(permissionCode) {
  return (req, res, next) => {
    if (!req.currentUser) {
      return res.redirect("/login");
    }
    if (!req.permissions?.has(permissionCode)) {
      return
      res.status(403).render("errors/403", {
        title: req.t("errors.403.title")
      });
    }
    next();
  };
}
```

8.4. Interface d'administration

Une page `/users/permissions` peut afficher :

- la liste des permissions ;
- une matrice rôles x permissions ;
- une fiche utilisateur avec overrides individuels ;

- un journal des modifications.

Pour le POC, il faut commencer par la matrice rôles x permissions. Les overrides utilisateur peuvent venir ensuite.

9. Étape 8 - Interface Bootstrap / UI G2M

Même si l'interface actuelle n'est pas strictement Bootstrap dans tous les écrans, les vues doivent rester responsive et cohérentes avec le style G2M existant.

9.1. Vues à créer

```
views/auth/login.ejs views/auth/activate.ejs views/users/invitations/index.ejs
views/users/invitations/form.ejs views/users/permissions/index.ejs
views/errors/403.ejs
```

9.2. Vues à adapter

```
views/users/index.ejs views/users/show.ejs views/users/form.ejs
views/partials/header.ejs
```

9.3. Messages i18n à ajouter

Les clés suivantes devront être ajoutées progressivement dans `locales/fr.json`, `locales/en.json`, `locales/es.json` :

- `auth.login.title` ;
- `auth.login.email` ;
- `auth.login.password` ;
- `auth.login.submit` ;
- `auth.login.error` ;
- `auth.login.genericNotice` ;
- `auth.logout` ;
- `activation.title` ;
- `activation.password` ;
- `activation.confirmPassword` ;
- `activation.submit` ;
- `activation.invalidToken` ;
- `activation.success` ;
- `users.invitations.title` ;
- `users.invitations.new` ;
- `users.invitations.status.invite` ;
- `users.invitations.status.activee` ;
- `permissions.title`.

9.4. Ergonomie recommandée

Liste des invitations :

- email ;
- nom complet ;
- rôle ;
- mission ou zone ;
- statut ;
- expiration ;
- actions.

Formulaire invitation :

- nom ;
- prénoms ;
- email ;
- rôle ;
- zone ;
- mission ;
- durée de validité ;
- bouton "Créer l'invitation".

Page activation :

- rappel de l'email ;
- champ mot de passe ;
- confirmation ;
- message de politique minimale ;
- bouton "Activer mon compte".

Page connexion :

- email ;
- mot de passe ;
- message sobre en cas d'échec ;
- message sobre lorsqu'un email invité déclenche l'envoi du lien d'activation ;
- lien éventuel vers assistance administrateur, pas vers inscription libre.

10. Étape 9 - Scénario de tests manuels

10.1. Création d'une invitation

25. Se connecter comme administrateur.
26. Ouvrir Paramétrages > Utilisateurs > Invitations.

27. Créer une invitation pour `test.invite@example.org`.
28. Vérifier que le statut est `invité`.
29. Vérifier qu'un token hashé est stocké, mais jamais le token en clair.
30. Vérifier qu'un log `user.invitation_created` est présent.

10.2. Demande de lien depuis la page de connexion

31. Ouvrir `/login`.
32. Saisir `test.invite@example.org`.
33. Laisser le mot de passe vide ou saisir une valeur quelconque.
34. Soumettre.
35. Vérifier que G2M affiche un message sobre indiquant que les instructions seront envoyées si l'adresse est autorisée.
36. Vérifier que l'email d'activation est envoyé ou, en développement, que le lien est écrit dans les logs serveur.
37. Vérifier qu'aucun compte actif n'est créé tant que le lien n'est pas utilisé.
38. Vérifier qu'un log `activation.email_sent` est présent.

10.3. Activation avec token valide

39. Ouvrir le lien `/activation/<token>`.
40. Vérifier que la page affiche l'email invité.
41. Saisir un mot de passe valide.
42. Soumettre.
43. Vérifier la redirection vers `/login`.
44. Vérifier que le compte est `actif`.
45. Vérifier que `email_verified = 1`.
46. Vérifier que `activated_at` et `used_at` sont renseignés.

10.4. Connexion réussie

47. Ouvrir `/login`.
48. Saisir email et mot de passe.
49. Vérifier que la connexion aboutit.
50. Vérifier la présence du cookie JWT `HttpOnly`.
51. Vérifier que `last_login` est mis à jour.

10.5. Connexion avec compte suspendu

52. Suspendre l'utilisateur depuis l'administration.
53. Tenter une connexion.
54. Vérifier que la connexion est refusée.

55. Vérifier que le message reste sobre.
56. Vérifier le log d'échec.

10.6. Activation avec token expiré ou déjà utilisé

57. Ouvrir un lien expiré.
58. Vérifier le message "Lien invalide ou expiré".
59. Ouvrir un lien déjà utilisé.
60. Vérifier le même message.
61. Vérifier qu'aucun nouveau mot de passe n'est accepté.

11. Ordre d'implémentation recommandé

Lot 1 - Socle base de données

Fichiers à modifier ou créer :

- `config/database.js`
- `models/User.js`
- `models/UserInvitation.js`
- `models/ActivationToken.js`
- `models/AuditLog.js`
- `tests/app.test.js`

Objectif : schéma stable, tests de création invitation et token.

Lot 2 - Invitations administrateur

Fichiers à modifier ou créer :

- `routes/userRoutes.js` ou `routes/adminUserRoutes.js`
- `controllers/userInvitationController.js`
- `views/users/invitations/index.ejs`
- `views/users/invitations/form.ejs`
- `locales/fr.json`
- `locales/en.json`
- `locales/es.json`

Objectif : créer et lister les invitations.

Lot 3 - Activation

Fichiers à modifier ou créer :

- `routes/authRoutes.js`
- `controllers/activationController.js`

- `views/auth/activate.ejs`
- `services/tokenService.js`
- `services/passwordService.js`
- `services/activationMailService.js`
- `services/mailService.js`

Objectif : envoyer ou renvoyer un lien d'activation et activer un compte à partir d'un token valide.

Lot 4 - Connexion et protection

Fichiers à modifier ou créer :

- `controllers/authController.js`
- `middlewares/authMiddleware.js`
- `views/auth/login.ejs`
- `app.js`
- `package.json`

Objectif : connexion JWT, cookie sécurisé, `requireAuth`.

Le contrôleur de connexion devra aussi gérer le cas particulier des emails invités non activés : recherche de l'invitation, génération ou réutilisation d'un token valide, envoi du mail d'activation, puis affichage d'un message sobre.

Lot 5 - Habilitations

Fichiers à modifier ou créer :

- `models/Permission.js`
- `services/permissionService.js`
- `middlewares/permissionMiddleware.js`
- `controllers/permissionController.js`
- `views/users/permissions/index.ejs`

Objectif : droits par rôle puis overrides utilisateur.

12. Extraits de code illustratifs

12.1. Service de mot de passe

```
const bcrypt = require("bcrypt"); const SALT_ROUNDS = 12; async function
hashPassword(password) { return bcrypt.hash(password, SALT_ROUNDS); } async
function verifyPassword(password, hash) { if (!hash) { return false; }
return bcrypt.compare(password, hash); } module.exports = { hashPassword,
verifyPassword };
```

12.2. Service de token

```
const crypto = require("node:crypto"); function generateToken() { return
crypto.randomBytes(32).toString("hex"); } function hashToken(token) { return
```

```
crypto.createHash("sha256").update(token).digest("hex"); } module.exports = {
generateToken, hashToken };
```

12.3. Contrôleur d'activation simplifié

```
const UserInvitation = require("../models/UserInvitation"); const ActivationToken =
require("../models/ActivationToken"); const User = require("../models/User"); const {
hashPassword } = require("../services/passwordService"); const { hashToken } =
require("../services/tokenService"); exports.activate = async (req, res) => { const
tokenHash = hashToken(req.params.token); const activation =
ActivationToken.findValidByHash(tokenHash); if (!activation) { return
res.status(400).render("auth/activate", { title: req.t("activation.title"),
error: req.t("activation.invalidToken") }); } if (req.body.password !==
req.body.password_confirm || req.body.password.length < 10) { return
res.status(400).render("auth/activate", { title: req.t("activation.title"),
error: req.t("activation.invalidPassword") }); } const invitation =
UserInvitation.findById(activation.invitation_id); const passwordHash = await
hashPassword(req.body.password); const user =
User.activateFromInvitation(invitation, passwordHash);
ActivationToken.markUsed(activation.id, user.id);
UserInvitation.markActivated(invitation.id); return
res.redirect("/login?activated=1"); };
```

12.4. Détection d'une invitation depuis la connexion

```
const User = require("../models/User"); const UserInvitation =
require("../models/UserInvitation"); const { sendActivationLink } =
require("../services/activationMailService"); const { verifyPassword } =
require("../services/passwordService"); exports.login = async (req, res) => { const
email = req.body.email?.trim().toLowerCase(); const password = req.body.password ||
""; const genericMessage = req.t("auth.login.genericNotice"); const user =
User.findAuthByEmail(email); if (!user) { const invitation =
UserInvitation.findValidPendingByEmail(email); if (invitation) { await
sendActivationLink(invitation); } return res.render("auth/login", {
notice: genericMessage }); } if (user.statut !== "actif" ||
!user.email_verified) { return res.status(403).render("auth/login", { error:
req.t("auth.login.error") }); } const validPassword = await
verifyPassword(password, user.password_hash); if (!validPassword) { return
res.status(401).render("auth/login", { error: req.t("auth.login.error") }); }
// Génération JWT + cookie HttpOnly, puis redirection. };
```

13. Points de vigilance

- Ne pas exposer le token d'activation dans les logs.
- Ne jamais stocker le token en clair.
- Ne pas laisser un utilisateur `invite` se connecter.
- Ne pas confondre le statut d'invitation et le statut du compte.
- Prévoir une migration douce des statuts existants.
- Protéger les routes administrateur dès que l'authentification existe.
- Tester les rôles avant d'activer des restrictions fortes sur toute l'application.
- Préserver les comptes existants pendant la migration.

14. Conclusion

G2M dispose déjà d'une base utile pour la gestion administrative des utilisateurs. Le besoin cible ne nécessite pas une réécriture complète du module, mais une extension structurée autour des invitations, de l'activation, de l'authentification et des habilitations.

La stratégie recommandée est progressive : d'abord stabiliser le modèle de données et les invitations, puis ajouter l'activation, ensuite la connexion, et enfin les permissions fines. Cette démarche limite les risques de régression tout en posant une architecture compatible avec PostgreSQL/PostGIS et les futures exigences de sécurité de production.